

```

maxFollow <- 84
cens.type <- "weibull"

# m1 -censoring adjustment
muC.adj <- log(1.5)
k.z3 <- 1
k.treat <- 0.9

z1_frac <- 0.25 # Default model index 'm1' (The 1st quartile of z1=er)
pH_super <- 0.125 # non-NULL re-defines z1_frac

if (is.null(pH_super)) {
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_frac)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_frac)))
  cat("Underlying pH_super", c(pH_check), "\n")
}
# pH_super specified If pH_super then override z1_frac and find z1_frac to
# yield pH_super

if (!is.null(pH_super)) {
  # Approximate Z1 quantile to yield pH proportion
  z1_q <- uniroot(propH.obj4, c(0, 1), tol = 1e-04, pH.target = pH_super)$root
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_q)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_q)))
  cat("pH", c(pH_check), "\n")
  rel_error <- (pH_super - pH_check)/pH_super
  if (abs(rel_error) >= 0.1)
    stop("pH_super approximation relative error exceeds 10%")
  z1_frac <- z1_q
  cat("Underlying pH_super", c(pH_check), "\n")
}

## pH 0.122449
## Underlying pH_super 0.122449

# Bootstrap on log(hr) scale converted to HR (est.loghr=TRUE & est.scale='hr')
est.loghr <- TRUE
est.scale <- "hr"
t.start.all <- proc.time()[3]

##### Forest search criteria
hr.threshold <- 1.25 # Initial candidates
hr.consistency <- 1 # Candidates for many splits
pconsistency.threshold <- 0.9
stop.threshold <- 0.95
maxk <- 2
nmin.fs <- 60
pstop_futile <- 0.5
# Limit timing for forestsearch
max.minutes <- 3
m1.threshold <- Inf # Turning this off (Default)
# pconsistency.threshold<-0.70 # Minimum threshold (will choose max among
# subgroups satisfying)

```

```

fs.splits <- 400 # How many times to split for consistency
# vi is % factor is selected in cross-validation --> higher more important
vi.grf.min <- 0.2
# Null, turns off grf screening
d.min <- 10 # Min number of events for both arms (d0.min=d1.min=d.min)
# default=5 Virtual twins analysis Counter-factual difference (C-E) >=
# vt.threshold Large values in favor of C (control)
vt.threshold <- 0.225 # For VT delta
treat.threshold <- 0

maxdepth <- 2
n.min <- 60
ntree <- 1000
# GRF criteria
dmin.grf <- 12 # For GRF delta
# Note: For CRT this represents dmin.grf/2 RMS for control (-dmin.grf/2 for
# treatment)
frac.tau <- 0.6

outcome.name <- c("y.sim")
event.name <- c("event.sim")
id.name <- c("id")
treat.name <- c("treat")

cox.formula.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat"))
cox.formula.adj.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5"))

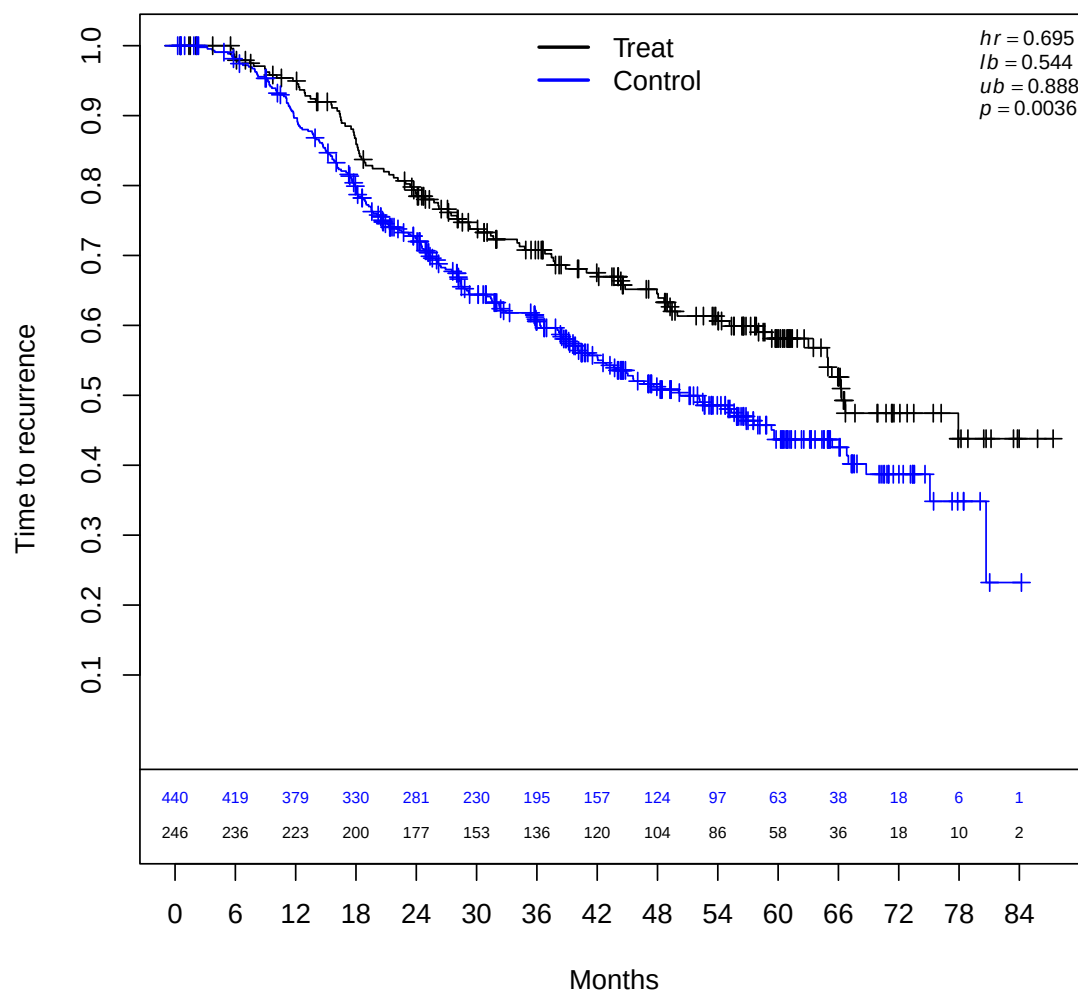
# Bootstrap on log(hr) scale converted to HR (est.loghr=TRUE & est.scale='hr')
est.loghr <- TRUE
est.scale <- "hr"

out.loc <- NULL
model.index <- "m4a"
file.index <- "v0"

mod.harm <- "alt"
hrH.target <- 2

N <- 700
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, model.index = "v0",
  hrH.target = hrH.target, cens.type = cens.type, out.loc = out.loc, file.index = file.index,
  details = TRUE, parms_torand = FALSE)

```



```
## Super-population empirical harm and non-harm hazard ratios= 2.000007 0.6466405
## Causal HR (empirical ITT)= 0.7057463
```

```
n_add_noise <- 3
sim <- 1

dgm <- this.dgm$dgm

xx <- sim_aftm4_gbsg(dgm = dgm, n = N, maxFollow = maxFollow, muC.adj = muC.adj,
  simid = sim)

x <- copy(xx)

if (n_add_noise == 0) {
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade")
}

if (n_add_noise == 5) {
```

```

    set.seed(8316951 + 1000 * sim)
    # Add 5 noise
    x$noise1 <- rnorm(N)
    x$noise2 <- rnorm(N)
    x$noise3 <- rnorm(N)
    x$noise4 <- rnorm(N)
    x$noise5 <- rnorm(N)
    confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade", "noise1",
                          "noise2", "noise3", "noise4", "noise5")
  }

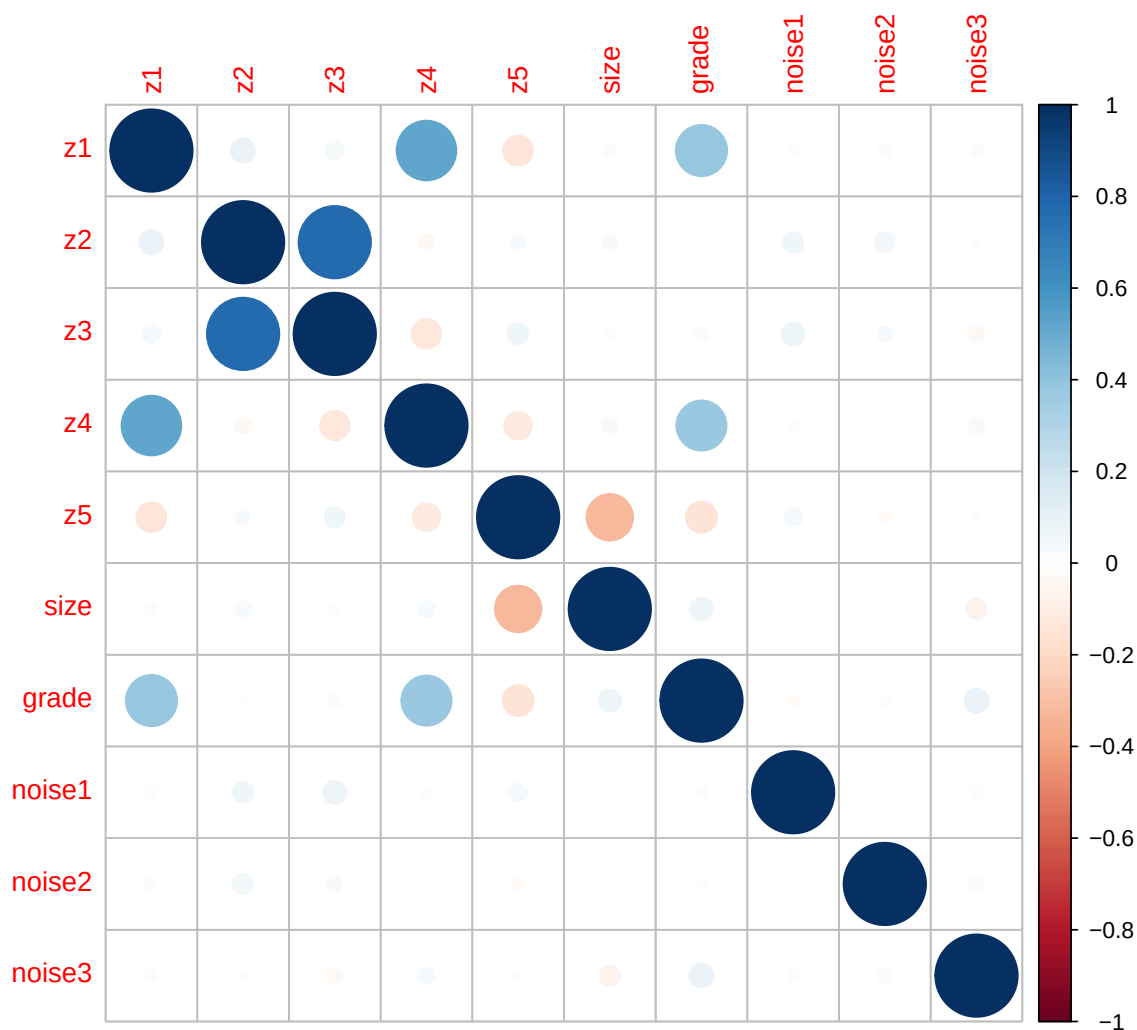
  if (n_add_noise == 3) {
    set.seed(8316951 + 1000 * sim)
    # Add 3 noise
    x$noise1 <- rnorm(N)
    x$noise2 <- rnorm(N)
    x$noise3 <- rnorm(N)
    confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade", "noise1",
                          "noise2", "noise3")
  }

  cox.formula.check <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5+size+grade+noise1+
  coxph(cox.formula.check, data = x)

## Call:
## coxph(formula = cox.formula.check, data = x)
##
##              coef exp(coef) se(coef)      z      p
## treat   -0.190994  0.826138  0.106954 -1.786  0.07414
## v11      0.646240  1.908352  0.139462  4.634 3.59e-06
## v21     -0.644920  0.524705  0.199045 -3.240  0.00119
## v31      0.910511  2.485592  0.202101  4.505 6.63e-06
## v41      0.601285  1.824462  0.134425  4.473 7.71e-06
## v51     -0.823439  0.438920  0.112756 -7.303 2.82e-13
## size     0.003801  1.003809  0.003565  1.066  0.28623
## grade   -0.042503  0.958388  0.100926 -0.421  0.67366
## noise1  -0.046821  0.954258  0.054690 -0.856  0.39193
## noise2   0.130029  1.138861  0.054183  2.400  0.01640
## noise3   0.027800  1.028190  0.052403  0.531  0.59576
##
## Likelihood ratio test=188.3 on 11 df, p=< 2.2e-16
## n= 700, number of events= 365

library(corrplot)
Zm <- cor(x[, c(confounders.name)])
corrplot(Zm)

```



```

use_lasso <- TRUE
use_grf <- FALSE

conf_force <- NULL
grf_depth <- 2

fs.est <- forestsearch(df.analysis = x, Allconfounders.name = confounders.name, details = TRUE,
  use_lasso = use_lasso, use_grf = use_grf, conf_force = conf_force, grf_depth = grf_depth,
  outcome.name = outcome.name, treat.name = treat.name, event.name = event.name,
  id.name = id.name, n.min = nmin.fs, hr.threshold = hr.threshold, hr.consistency = hr.consistency,
  fs.splits = fs.splits, d0.min = d.min, d1.min = d.min, pstop_futile = pstop_futile,
  pconsistency.threshold = pconsistency.threshold, stop.threshold = stop.threshold,
  max.minutes = max.minutes, maxk = maxk, by.risk = 12, plot.sg = TRUE, vi.grf.min = vi.grf.min)

## # of continuous/categorical characteristics 4 6
## Continuous characteristics: size noise1 noise2 noise3
## Categorical characteristics: z1 z2 z3 z4 z5 grade
## Cox-LASSO selected: z1 z2 z3 z4 z5 size grade noise1 noise2 noise3
## Cox-LASSO not selected:

```

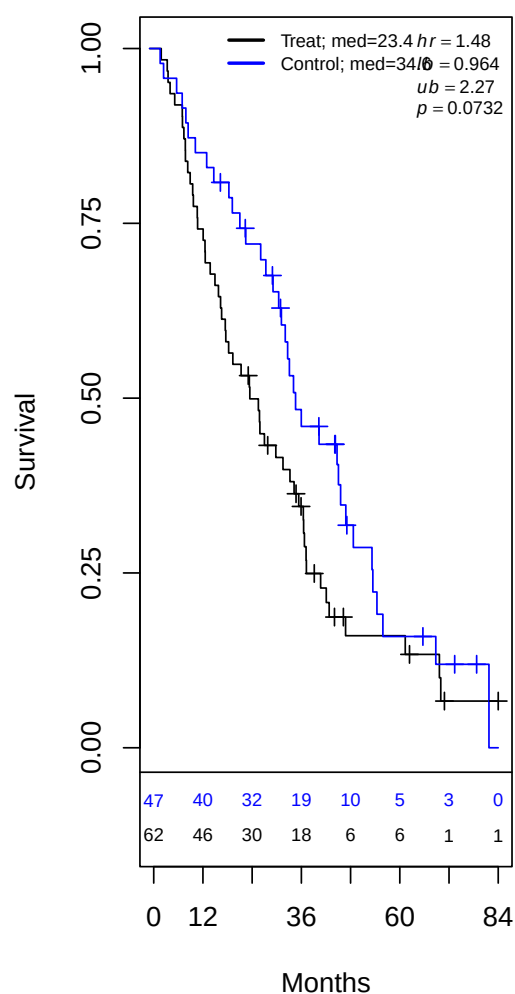
```

## Median cuts after Lasso size noise1 noise2 noise3
## Categorical after Lasso z1 z2 z3 z4 z5 grade
## # of candidate subgroup factors= 10
## [1] "size <= median(size)"      "noise1 <= median(noise1)"
## [3] "noise2 <= median(noise2)"  "noise3 <= median(noise3)"
## [5] "z1"                        "z2"
## [7] "z3"                        "z4"
## [9] "z5"                        "grade"
## Confounders per grf screening q6 q7 q10 q4 q5 q9 q3 q1 q8 q2
##      vi.cs FSconfounders.name      vi.cs
## 6  0.06295282      q6 0.27125668
## 7  0.05971892      q7 0.12647366
## 10 0.06465095     q10 0.11403343
## 4  0.09544804      q4 0.09544804
## 5  0.07766086      q5 0.07766086
## 9  0.27125668      q9 0.06536939
## 3  0.12647366      q3 0.06465095
## 1  0.06243525      q1 0.06295282
## 8  0.06536939      q8 0.06243525
## 2  0.11403343      q2 0.05971892
## Number of unique levels (L) and possible subgroups= 21 2097151
## Number of possible subgroups (in millions)= 2.097151
## # of subgroups based on # variables > k.max and excluded (per million) 2.09692
## k.max= 2
## Events criteria for control,exp= 10 10
## # of subgroups with events less than criteria: control, experimental 26 22
## # of subgroups with sample size less than criteria 30
## # of subgroups meeting all criteria = 200
## # of subgroups fitted (Cox model estimable) = 200
## *Subgroup Searching Minutes=* 0.1768333
## Number of subgroups meeting HR threshold 10
## # of candidate subgroups (meeting HR criteria) = 10
## Subgroups (1st 10) meeting overall screening thresholds (HR, m1) sorted by HRs= Inf
##      K  n   E d1   m1   m0  HR L(HR) U(HR) q6.0 q6.1 q7.0 q7.1 q10.1 q10.2
## 1: 2  77  30 15 64.35   NA 1.61 0.78 3.30  0  0  0  0  0  1  0
## 2: 2  63  22 12 68.24   NA 1.57 0.67 3.65  0  0  0  0  0  1  0
## 3: 2  84  67 30 26.99 39.47 1.53 0.94 2.50  0  0  0  0  0  0  0
## 4: 2  85  74 45 19.29 32.07 1.51 0.94 2.41  0  0  0  1  0  0
## 5: 1  89  37 19 63.58   NA 1.50 0.78 2.87  0  0  0  0  1  0
## 6: 2 109  88 52 23.44 34.58 1.48 0.96 2.27  0  1  0  0  0  0
## 7: 2  95  76 38 24.55 40.31 1.37 0.87 2.16  0  0  0  0  0  0
## 8: 2 118  87 49 23.39 30.47 1.36 0.89 2.09  0  0  0  1  0  0
## 9: 2 186 104 57 42.07 55.88 1.36 0.92 2.00  0  1  0  0  0  0
## 10: 2 149  88 47 37.41 54.22 1.29 0.85 1.97  0  0  0  1  0  0
##      q10.3 q4.0 q4.1 q5.0 q5.1 q9.0 q9.1 q3.0 q3.1 q1.0 q1.1 q8.0 q8.1 q2.0 q2.1
## 1:  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0
## 2:  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0
## 3:  0  0  0  0  0  1  0  0  0  0  0  0  0  0  1
## 4:  0  0  0  0  0  1  0  0  0  0  0  0  0  0  0
## 5:  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
## 6:  0  0  0  0  0  1  0  0  0  0  0  0  0  0  0
## 7:  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0
## 8:  0  0  0  0  0  0  0  0  0  0  0  0  1  0  0
## 9:  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0
## 10: 0  0  0  0  0  0  0  0  0  1  0  0  0  0  0

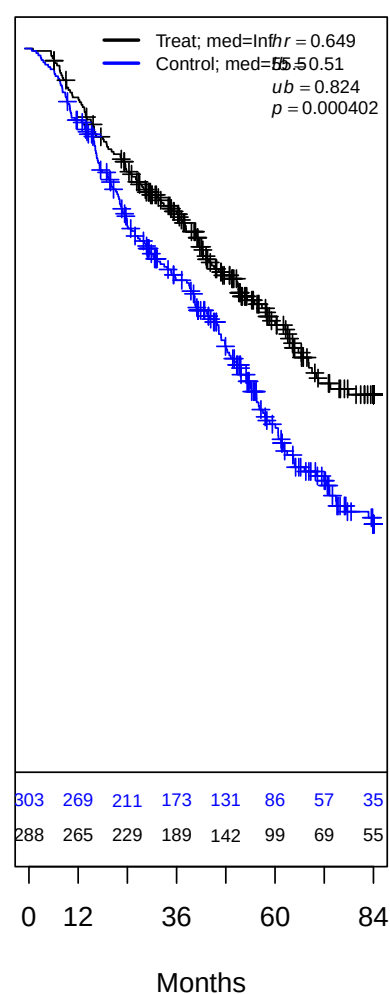
```

```
## Consistency 0.7875
## Consistency 0.66
## Consistency 0.9025
## [1] "q5.1" "q2.1"
## [1] "{z1}"                                "{noise1 <= median(noise1)}"
## # of splits= 400
## Model, % Consistency Met= {z1} {noise1 <= median(noise1)} 0.9025
## Consistency 0.8975
## Consistency 0.785
## Consistency 0.955
## [1] "q6.1" "q5.1"
## [1] "{z2}" "{z1}"
## # of splits= 400
## Model, % Consistency Met= {z2} {z1} 0.955
## Number of subgroups meeting consistency criteria=
## p.consistency Nsg group.id m.index K M.1 M.2
## 1: 0.9550 109 4 6 2 {z2} {z1}
## 2: 0.9025 84 10 3 2 {z1} {noise1 <= median(noise1)}
```

{z2} & {z1}



Complement



```

## [1] "{z2}" "{z1}"
## % consistency criteria met= 0.955
## SG focus= hr
## Subgroup Consistency Minutes= 0.1304667
## Subgroup found
## Minutes overall= 0.32705

grf.est <- grf.subg.harm.survival(data = x, confounders.name = confounders.name,
  outcome.name = outcome.name, event.name = event.name, id.name = id.name, treat.name = treat.name,
  n.min = n.min, dmin.grf = dmin.grf, frac.tau = 1, maxdepth = grf_depth, details = TRUE)

## tau, maxdepth= 77.92977 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2   -10.535829   325.000000   2.834699    10.535829   325.000000
## 2         3    2.727493   375.000000   2.515591    -2.727493   375.000000
## 3         4   -17.261313   150.000000   4.120255    17.261313   150.000000
## 4         5    7.069757   152.000000   3.966572    -7.069757   152.000000
## 5         6    8.102285   146.000000   4.006352    -8.102285   146.000000
## 6         7   -8.212993   252.000000   3.113917     8.212993   252.000000
##   treated.se      diff Nsg depth
## 1  2.834699 -21.071658 325     1
## 2  2.515591   5.454986 375     1
## 3  4.120255 -34.522626 150     2
## 4  3.966572  14.139513 152     2
## 5  4.006352  16.204571 146     2
## 6  3.113917 -16.425985 252     2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 5         6    8.102285   146.000000   4.006352    -8.102285   146.000000
##   treated.se      diff Nsg depth
## 5  4.006352  16.20457 146     2
## [1] "noise2 <= -0.12" "z2 <= 0"          "noise1 <= -0.42"
## [1] "noise1 <= -0.419923509251078"

# plot(grf.est$tree)

library(doParallel)
registerDoParallel(parallel::detectCores(logical = FALSE))

cox.formula.boot <- cox.formula.sim
est.loghr <- TRUE
max.minutes <- 7
# Suggest running 50, first ... to get timing estimate

NB <- 1000
df_boot_analysis <- fs.est$df.est
##### est.loghr=T --> log(HR) scale
fitH <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 0), cox.formula = cox.formula,
  est.loghr = est.loghr)
H_obs <- fitH$est_obs # log(hr) scale
seH_obs <- fitH$se_obs
# Hc observed estimates
fitHc <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 1), cox.formula = cox.formula,
  est.loghr = est.loghr)
Hc_obs <- fitHc$est_obs
seHc_obs <- fitHc$se_obs
rm("fitH", "fitHc")

```

```

Ystar_mat <- bootYstar({
  ystar <- get_Ystar(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
# Check dimension
if (dim(Ystar_mat)[1] != NB | dim(Ystar_mat)[2] != nrow(df_boot_analysis)) stop("Dimension of Ystar_

tB.start <- proc.time()[3]
# Bootstraps
resB <- bootPar({
  ans <- fsboot_forparallel(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
tB.now <- proc.time()[3]
tB.min <- (tB.now - tB.start)/60

doParallel::stopImplicitCluster()

cat("Minutes for Boots", c(NB, tB.min), "\n")

## Minutes for Boots 1000 4.129917

cat("Projection per 100", c(tB.min * (100/NB)), "\n")

## Projection per 100 0.4129917

cat("Propn bootstrap subgroups found =", c(sum(!is.na(resB$H_biasadj_1))/NB), "\n")

## Propn bootstrap subgroups found = 0.953

# How many timed out
cat("Number timed out=", c(sum(is.na(resB$H_biasadj_1) & resB$tmins_search > max.minutes)),
  "\n")

## Number timed out= 0

H_estimates <- get_dfRes(Hobs = H_obs, seHobs = seH_obs, H1_adj = resB$H_biasadj_1,
  ystar = Ystar_mat, cov_method = "standard", cov_trim = 0.05)

Hc_estimates <- get_dfRes(Hobs = Hc_obs, seHobs = seHc_obs, H1_adj = resB$Hc_biasadj_1,
  ystar = Ystar_mat, cov_method = "standard", cov_trim = 0.05)

print(H_estimates)

##           H0          sdH0  H0_lower H0_upper          H1          sdH1  H1_lower H1_upper
## 1: 1.478681 0.3227903 0.9639631 2.268237 1.233028 0.09980086 1.052149 1.445004

print(Hc_estimates)

##           H0          sdH0  H0_lower  H0_upper          H1          sdH1  H1_lower
## 1: 0.6485153 0.07935847 0.5102219 0.8242924 0.6605546 0.04425006 0.5792784
##           H1_upper
## 1: 0.7532343

bootit <- list(H_estimates = H_estimates, Hc_estimates = Hc_estimates)
ans3 <- fsBoot.parallel.out(df = df_boot_analysis, FSboots = bootit, cox.formula.boot = cox.formula.
  est.loghr = est.loghr, sim_number = sim)

# Revise path accordingly
# save(dgm,x,y,H_estimates,Hc_estimates,ans3,file='output/sim_ex1.Rdata')

```

```

df_pred <- fs.est$df.predict
df <- df_pred
tte.name <- "y.sim"
event.name <- "event.sim"
treat.name <- "treat"
byrisk <- 9
risk.cex <- 0.6
legend.cex <- 0.75
details <- TRUE
ylab <- "Survival"
# Add all events
evs <- sort(unique(df$y.sim[which(df$event.sim == 1)]))
tpoints.add <- c(-1, evs, max(df$y.sim))
pcens <- 100 * round(mean(1 - df$event.sim), 2)
nn <- length(df$y.sim)
nH_true <- sum(x$flag.harm)
nHc_true <- sum(1 - x$flag.harm)
nH_est <- sum(1 - df$treat.recommend)
kH <- with(df, sum(treat.recommend == 0 & flag.harm == 1))
sens_H <- kH/nH_true
spec_H <- kH/nH_est

nHc_est <- sum(df$treat.recommend)
kHc <- with(df, sum(treat.recommend == 1 & flag.harm == 0))

pH_super <- 100 * round(mean(dgm$df.super_rand$flag.harm), 2)

# ans3b<-format(ans3,digits=4,drop0trailing=TRUE) ans3<-as.data.frame(ans3b)

ans3 <- round(ans3, 3)

# ITT estimates
resITT <- c(ans3[, c("H_itt", "H_itt_lower", "H_itt_upper")], NA, NA)

# Knowing subgroup a-priori Returns coverage status for hr.H.true coverage for
# a-prior is NA
temp <- getci_Cox(df = as.data.frame(ans3), est = "H_true", se = "seH_true", target = unlist(dgm["hr.H
resH_apriori <- c(ans3[, c("H_true")], temp$lb, temp$ub, NA, temp$cover)
##### Observed
temp <- getci_Cox(df = as.data.frame(ans3), est = "H_obs", se = "seH_obs", target = unlist(dgm["hr.H
# Cover a-priori ?
estH_apriori <- ans3[, c("H_true")]
cover2 <- ifelse(estH_apriori >= c(temp$lb) & estH_apriori <= c(temp$ub), 1, 0)
resH_obs <- c(ans3[, c("H_obs")], temp$lb, temp$ub, cover2, temp$cover)
# Bias-corrected
temp <- getci_Cox(df = as.data.frame(ans3), est = "H1.bc", se = "seH1.bc", target = unlist(dgm["hr.H
# Cover a-priori ?
cover2 <- ifelse(estH_apriori >= c(temp$lb) & estH_apriori <= c(temp$ub), 1, 0)
resH_bc <- c(ans3[, c("H1.bc")], temp$lb, temp$ub, cover2, temp$cover)
#####
resH <- rbind(resH_apriori, resH_obs, resH_bc)
##### Repeat above for Hc
temp <- getci_Cox(df = as.data.frame(ans3), est = "Hc_true", se = "seHc_true", target = unlist(dgm["
resH_apriori <- c(ans3[, c("Hc_true")], temp$lb, temp$ub, NA, temp$cover)
temp <- getci_Cox(df = as.data.frame(ans3), est = "Hc_obs", se = "seHc_obs", target = unlist(dgm["hr

```

```

estH_apriori <- ans3[, c("Hc_true")]
cover2 <- ifelse(estH_apriori >= c(temp$lb) & estH_apriori <= c(temp$ub), 1, 0)
resH_obs <- c(ans3[, c("Hc_obs")], temp$lb, temp$ub, cover2, temp$cover)
temp <- getci_Cox(df = as.data.frame(ans3), est = "Hc1.bc", se = "seHc1.bc", target = unlist(dgm["hr
cover2 <- ifelse(estH_apriori >= c(temp$lb) & estH_apriori <= c(temp$ub), 1, 0)
resH_bc <- c(ans3[, c("Hc1.bc")], temp$lb, temp$ub, cover2, temp$cover)
#####
resHc <- rbind(resH_apriori, resH_obs, resH_bc)
#####

res <- rbind(resITT, resH, resHc)
resf <- as.data.frame(res)
colnames(resf) <- c("HR Estimate", "Lower", "Upper", "C(true)", "C(fixed)")

# Per Keaven's suggestion, remove C(.) and include # of subjects

resf_new <- resf[, -c(4, 5)]
# Add sample size columns
na <- c(nn, nH_true, nH_est, nH_est, nHc_true, nHc_est, nHc_est)
ncorrect <- c(NA, NA, kH, kH, NA, kHc, kHc)
nnew <- cbind(na, ncorrect)
resf_new <- cbind(resf_new, nnew)

rm("resf")

colnames(resf_new) <- c("HR Estimate", "Lower", "Upper", "$\\#$ Subjects", "$\\#$ Correct")
resf <- resf_new

# rnH<-c('L\\hat\\theta(H)L', 'L\\hat\\theta(\\hat{H})L', 'L\\hat\\theta^{*}(\\hat{H})L')
# rnHc<-c('L\\hat\\theta(H^{c})L', 'L\\hat\\theta(\\hat{H}^{c})L', 'L\\hat\\theta^{*}(\\hat{H}^{c})L')
# rnItt<-c('L\\hat\\theta(ITT)L')

rnH <- c("H true", "$\\hat{H}$", "$\\hat{H}_{bc}$")
rnHc <- c("$H^{c}$ true", "$\\hat{H}^{c}$", "$\\hat{H}^{c}_{bc}$")
rnItt <- c("ITT")

rownames(resf) <- c(rnItt, rnH, rnHc)

# Describing CIs
Hstat <- c(unlist(resH[1, ]))[c(1, 2, 3)]
Hstat <- round(Hstat, 2)
a <- paste0(Hstat[1], " (95% CI=")
a <- paste0(a, Hstat[2])
a <- paste0(a, ",")
a <- paste0(a, Hstat[3])
a <- paste0(a, ")")
Hstat_true <- c(a)

Hstat <- c(unlist(resH[2, ]))[c(1, 2, 3)]
Hstat <- round(Hstat, 2)
a <- paste0(Hstat[1], " (95% CI=")
a <- paste0(a, Hstat[2])
a <- paste0(a, ",")
a <- paste0(a, Hstat[3])
a <- paste0(a, ")")

```

Table 1 Simulated data example: Cox hazard ratio (hr) estimates for the ITT population and subgroups H and H^c . Cox model estimates are based on subgroups: H true (knowing the actual subgroup, a-priori); the estimated subgroup $\hat{H}$; and the bootstrap (1,000 resamples) bias-correction to $\hat{H}$ estimates, denoted $\hat{H}_{bc}$. Estimates for the complement H^c are defined analogously. The number of subjects in each population (# Subjects) and the number of subjects correctly classified (# Correct) in the true subgroups H and H^c , respectively, are listed.

	HR Estimate	Lower	Upper	# Subjects	# Correct
ITT	0.784	0.638	0.963	700	.
H true	1.506	0.941	2.409	85	.
$\hat{H}$	1.479	0.964	2.269	109	85
$\hat{H}_{bc}$	1.233	1.052	1.445	109	85
H^c true	0.665	0.527	0.839	615	.
$\hat{H}^c$	0.649	0.511	0.824	591	591
$\hat{H}_{bc}^c$	0.661	0.58	0.753	591	591

```
Hstat_est <- c(a)

Hstat <- c(unlist(resH[3, ])) [c(1, 2, 3)]
Hstat <- round(Hstat, 2)
a <- paste0(Hstat[1], " (95% CI=")
a <- paste0(a, Hstat[2])
a <- paste0(a, ",")
a <- paste0(a, Hstat[3])
a <- paste0(a, ")")
Hstat_bc <- c(a)
```